

BookNest

Full Stack Bookstore Web Application

Project Documentation

Full Stack Development with MERN

Prepared by: Manisha Baddireddy
Individual Project

1. Introduction

Project Title: BookNest – Full Stack Bookstore Web Application

Team Members: This is an individual project, designed, developed, and deployed independently by Manisha Baddireddy, who handled the complete full-stack development — frontend, backend, database design, and API integration.

2. Project Overview

Purpose:

BookNest is a full-stack online bookstore platform that connects readers with sellers through a centralized, admin-monitored marketplace. The goal of the project is to provide a simple and reliable system where users can discover and purchase books, sellers can manage their own book inventory, and administrators can oversee the entire platform through a single dashboard.

Features:

User Module

- Sign up and login
- Browse all available books
- Search books by Title, Author, or Genre
- Purchase books and enter a delivery address
- View order history and delivery dates

Seller Module

- Sign up and login
- Add new books to the store
- Upload book cover images
- View and delete listed books
- View customer orders

Admin Module

- Sign up and login
- Dashboard with platform statistics
- View and delete users
- View and delete sellers
- View and delete books

3. Architecture

Frontend (React): The client is built using React along with React Router for page navigation and Bootstrap 5 for responsive UI styling. Axios is used to communicate with the backend REST APIs. The frontend is organized into role-based modules for Users, Sellers, and Admins, along with a shared Components folder for reusable UI elements such as navigation bars, cards, and forms.

Backend (Node.js & Express.js): The server is built using Node.js with the Express.js framework, exposing REST APIs for user, seller, and admin operations. Routes are separated by role (user,

seller, admin), and middleware handles authentication, authorization, and file uploads (book cover images).

Database (MongoDB): MongoDB is used as the database, with Mongoose as the ODM (Object Data Modeling) library to define schemas for Users, Sellers, Admins, Books, and Orders. Mongoose handles schema validation and structures the relationships between books, sellers, and the orders placed by users.

4. Setup Instructions

Prerequisites:

- Node.js (v18 or later)
- MongoDB (local instance or MongoDB Atlas)
- npm (Node Package Manager)
- Git

Installation:

1. Clone the repository:

```
git clone https://github.com/manishabaddireddy/booknest.git
cd booknest
```

2. Install backend dependencies:

```
cd server
npm install
```

3. Install frontend dependencies:

```
cd client
npm install
```

4. Configure environment variables in the server folder (e.g. **.env**) with values such as:

```
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret_key
PORT=5000
```

5. Start MongoDB (if running locally), then start the backend and frontend servers as described in Section 6.

5. Folder Structure

Client (React Frontend):

```
client/
├── src/
│   ├── Components/ # Shared/reusable UI components
│   ├── User/       # User-facing pages (browse, search, orders)
│   ├── Seller/     # Seller dashboard, add/manage books
│   └── Admin/      # Admin dashboard, manage users/sellers/books
```

Server (Node.js / Express Backend):

```
server/  
■■■ routes/  
    ■■■ user.js    # User signup, login, browse, orders  
    ■■■ seller.js  # Seller signup, login, book & order management  
    ■■■ admin.js   # Admin signup, login, platform management  
  
demo screenshots/ # UI screenshots showcasing the application
```

6. Running the Application

Backend — from the **server** directory:

```
npm start
```

Runs the Express server (default: <http://localhost:5000>).

Frontend — from the **client** directory:

```
npm start
```

Runs the React development server (default: <http://localhost:3000>). Once both servers are running, open the frontend URL in a browser to use the application.

7. API Documentation

All backend endpoints are prefixed with **/api** and organized by role. Search parameters can be appended to the books endpoint as query strings, e.g. **/api/user/books?title=**, **?author=**, or **?genre=**.

User APIs

Method	Endpoint	Description
POST	/api/user/signup	Register a new user
POST	/api/user/login	User login
GET	/api/user/books	Get all books (supports search filters)
POST	/api/user/order	Place an order
GET	/api/user/orders/:userId	Get a user's order history

Seller APIs

Method	Endpoint	Description
POST	/api/seller/signup	Register a new seller
POST	/api/seller/login	Seller login
POST	/api/seller/addbook	Add a new book with cover image
GET	/api/seller/mybooks/:sellerId	Get seller's listed books
DELETE	/api/seller/book/:id	Delete a book
GET	/api/seller/orders/:sellerId	Get orders for seller's books

Admin APIs

Method	Endpoint	Description
POST	/api/admin/signup	Register a new admin
POST	/api/admin/login	Admin login
GET	/api/admin/users	Get all registered users
GET	/api/admin/sellers	Get all registered sellers
GET	/api/admin/books	Get all books on the platform
DELETE	/api/admin/user/:id	Delete a user
DELETE	/api/admin/seller/:id	Delete a seller
DELETE	/api/admin/book/:id	Delete a book

8. Authentication

BookNest uses **JWT (JSON Web Tokens)** for authentication and **bcryptjs** for password security.

- During signup, user, seller, and admin passwords are hashed using bcryptjs before being stored in MongoDB.
- On login, credentials are verified against the hashed password, and a signed JWT is issued to the client.
- The JWT is sent by the client on subsequent requests (typically via an Authorization header) to access protected routes.
- The backend verifies the token on each protected request to authenticate the user, seller, or admin and authorize role-specific actions.
- Separate signup/login flows exist for each role (User, Seller, Admin), ensuring role-based access control across the platform.

9. User Interface

The interface is built with React and Bootstrap 5, providing a clean and responsive experience across the three role-based views:

- **User view:** Book browsing/search pages, book detail view, cart/checkout with delivery address entry, and an order history page.
- **Seller view:** Dashboard to add books (with cover image upload), a listing of the seller's own books with delete options, and an orders view.
- **Admin view:** A statistics dashboard along with management tables for users, sellers, and books, each with delete actions.

Representative UI screenshots are provided in Section 11 (Screenshots / Demo).

10. Testing

As an individually developed project, testing was carried out primarily through manual functional testing during development, including:

- Manual end-to-end testing of signup/login flows for all three roles (User, Seller, Admin).
- Manual verification of book search filters (title, author, genre).
- Testing the order placement flow, including delivery address entry and order history retrieval.
- API testing of backend routes using tools such as Postman to verify request/response behavior.
- Cross-checking CRUD operations (add/view/delete) for books, users, and sellers from the admin dashboard.

11. Screenshots or Demo

Screenshots demonstrating the application's User, Seller, and Admin interfaces are available in the **"demo screenshots"** folder of the project repository:

<https://github.com/manishabaddireddy/booknest/tree/main/demo%20screenshots>

12. Known Issues

- No automated test suite (unit/integration tests) is currently included in the project.
- Payment gateway integration is not implemented; orders are recorded without real payment processing.
- Limited input validation and error handling on some forms could be strengthened.
- No password-reset / forgot-password flow is currently available for users, sellers, or admins.

13. Future Enhancements

- Integrate a real payment gateway (e.g., Razorpay/Stripe) for secure online transactions.
- Add order status tracking (e.g., Pending, Shipped, Delivered) with real-time updates.
- Implement ratings and reviews for books.
- Add pagination and advanced filtering/sorting for the book catalog.
- Add automated testing (unit and integration tests) for both frontend and backend.
- Introduce email notifications for order confirmation and delivery updates.
- Add a wishlist/favorites feature for users.